

HackMate

Campus Hackathon Team Discovery & Squad Management Platform

Complete Technical Stack Specification & Architectural Report

Project Name:	HackMate	Architecture:	Decoupled Multi-Tier REST
Backend Stack:	Java 17 • Javalin 5.6 • Jetty	Frontend Stack:	React 18 • Vite 5 • Tailwind • GSAP
Database:	MongoDB 7.0+ (NoSQL)	Security:	JWT (HMAC256) • BCrypt Hashing

1. System Architecture Overview

HackMate implements a high-performance decoupled multi-tier web architecture designed specifically for collegiate hackathon squad discovery. The architecture separates presentation concerns from business validation and persistence, communicating over a structured RESTful JSON contract.

Tier	Technologies Utilized	Architectural Responsibility
Client Tier (Frontend)	React 18, Vite 5, Tailwind CSS 3.4, GSAP 3.12, Fetch API	State-driven UI, real-time multi-criteria filtering, modal pitch submission, roster visualization.
Application Tier (REST API)	Java 17 LTS, Javalin 5.6.3, Embedded Jetty, Jackson Databind	HTTP routing, business rule enforcement, roster capacity calculations, CORS headers.
Security & Auth	Auth0 Java-JWT (HMAC256), jBCrypt 0.4, Javalin AuthFilter	Stateless session verification, password salting/hashing, protected endpoint access control.
Persistence Tier (Database)	MongoDB 7.0, MongoDB Java Sync Driver 4.11, mongo-java-server	Schema persistence for users, squads, and requests; atomic array mutations (\$push, \$set).

2. Detailed Technology Specifications

A. Frontend Layer (User Interface & Experience)

Component	Version	Role & Implementation Details
React	18.2.0	State-driven component islands (`TeamCard`, `Navbar`, `FilterBar`, `CreateTeamModal`, `ApplyModal`).
Vite	5.1.4	Next-generation build tool with lightning-fast Hot Module Replacement and production bundling.
Tailwind CSS	3.4.1	Utility-first responsive design with custom warm editorial palette and tactile elevation shadows.
GSAP	3.12.5	GreenSock Animation Platform for smooth entrance timelines, spring tab switching, and floating badges.
Fetch Client	Native ES6	Centralized `api.js` client handling JWT Bearer token injection and unified error serialization.

B. Backend Layer (REST API & Application Logic)

Component	Version	Role & Implementation Details
Java (JDK)	17 LTS	Core language runtime leveraging modern LTS performance, pattern matching, and strong typing.
Javalin	5.6.3	Lightweight, unopinionated microframework on embedded Jetty; sub-second cold starts, zero enterprise bloat.
Jackson Databind	2.16.1	High-performance JSON serialization and deserialization for all HTTP request/response payloads.
Apache Maven	3.9.6	Build lifecycle automation, dependency management, and production Shade Fat-JAR packaging.
SLF4J / Logger	2.0.12	Structured application logging for request profiling, database connection lifecycle, and errors.

C. Database Layer (Persistence & Schema Design)

Component	Version	Role & Implementation Details
MongoDB	7.0+	NoSQL Document Store storing JSON-native polymorphic records with embedded arrays.
MongoDB Java Driver	4.11.1	Official sync driver providing connection pooling, BSON filters, and atomic update operators (\$push).
mongo-java-server	1.45.0	In-memory wire-protocol mock enabling instant fallback execution for evaluation environments.

Primary MongoDB Collections:

- **users:** { _id, name, email, passwordHash, branch, year, skills: [String], createdAt }
- **teams:** { _id, title, hackathonName, description, skillsNeeded: [String], teamSize, createdBy, creatorName, status: 'OPEN'|'FULL', currentMembers: [{ userId, name, role }], createdAt }
- **join_requests:** { _id, teamId, teamTitle, applicantId, applicantName, applicantEmail, applicantBranch, applicantSkills: [String], message, status: 'PENDING'|'ACCEPTED'|'REJECTED', createdAt }

D. Security & Authentication Layer

Security Mechanism	Library / Standard	Implementation Details
JWT Session Tokens	Auth0 java-jwt 4.4.0	HMAC256 cryptographically signed tokens containing user ID & email; 24-hour validity.
Password Hashing	jBCrypt 0.4	Adaptive cryptographic one-way hashing with salt factor 10. Plaintext is never stored.
AuthFilter Middleware	Javalin Request Filter	Intercepts <code>/api/*</code> , verifies <code>Authorization: Bearer `</code> , and injects user context.

3. 4-Member Project Work Division & Presentation Scripts

Member	Assigned Functional Role	Core Technical Deliverables
Member 1 (Lead)	System Architecture & Backend REST	Javalin REST API routing, live demo coordination, controller logic, and overall system design.
Member 2	Frontend UI & GSAP Micro-Animations	React 18 components, Tailwind CSS styling system, modal state transitions, and GSAP timelines.
Member 3	Database & MongoDB Modeling	MongoDB schema design, polymorphic skill arrays, seed data generation, and atomic \$push updates.
Member 4	Authentication & Security Pipeline	BCrypt password salting/hashing, JWT HMAC256 token lifecycle, and Javalin AuthFilter route guards.

Individual Speaking Scripts for Viva / Presentation:

Member 1 (Introduction & Architecture):

"Good morning, Professor. We are presenting HackMate, a specialized campus hackathon squad discovery platform. Traditional WhatsApp groups suffer from unread message floods and unverified claims. HackMate provides verified student profiles, real-time roster meters, and structured 1-click pitch reviews using React, Java 17 Javalin REST APIs, and MongoDB."

Member 2 (Frontend & Animations):

"I developed the Frontend UI using React 18 and Tailwind CSS, creating modular components like TeamCard, FilterBar, and dynamic application modals. I also integrated GSAP for smooth entrance timelines and spring tab transitions between Sign In and Registration."

Member 3 (Database & Modeling):

"I designed the MongoDB database layer across users, teams, and join_requests. MongoDB was chosen because hackathon squads have dynamic skill arrays and nested rosters, allowing atomic updates with \$push without multi-table SQL JOIN overhead."

Member 4 (Authentication & Security):

"I implemented the security pipeline. Student passwords are salted and hashed using BCrypt. User sessions use stateless JWT tokens signed with HMAC256, and a global Javalin AuthFilter protects private endpoints."

4. Key Technical Justifications for Evaluators

Q1: Why use MongoDB instead of a Relational Database like MySQL?

Hackathon team postings are inherently polymorphic—each team requires arbitrary, dynamic arrays of technologies (e.g. ['React', 'FastAPI', 'PyTorch', 'Solidity']) and maintains an embedded list of accepted members with dynamic roles. In MongoDB, these are stored natively inside the squad document and queried directly. In MySQL, this would require 4 separate normalized tables (teams, skills, team_skills, team_members) joined on every card render, adding significant latency without relational benefits.

Q2: Why choose Javalin over Spring Boot?

Spring Boot introduces heavy reflection, extensive annotation scanning, and high memory overhead (~400MB+ heap). Javalin provides an ultra-lightweight, expressive REST routing model built directly on embedded Jetty with sub-second startup times (<1s) and explicit, transparent request lifecycles without hidden magic.

Q3: How does the system handle concurrent applications?

MongoDB executes atomic `\$push` and `\$set` operators on the team's `currentMembers` array. When a squad reaches its designated `teamSize`, the controller atomically flips the team status to `FULL`, preventing race conditions.